

Asynchronous data transfer

Daniele Riboni
Università degli Studi di Cagliari
Dept. of Mathematics and Computer Science

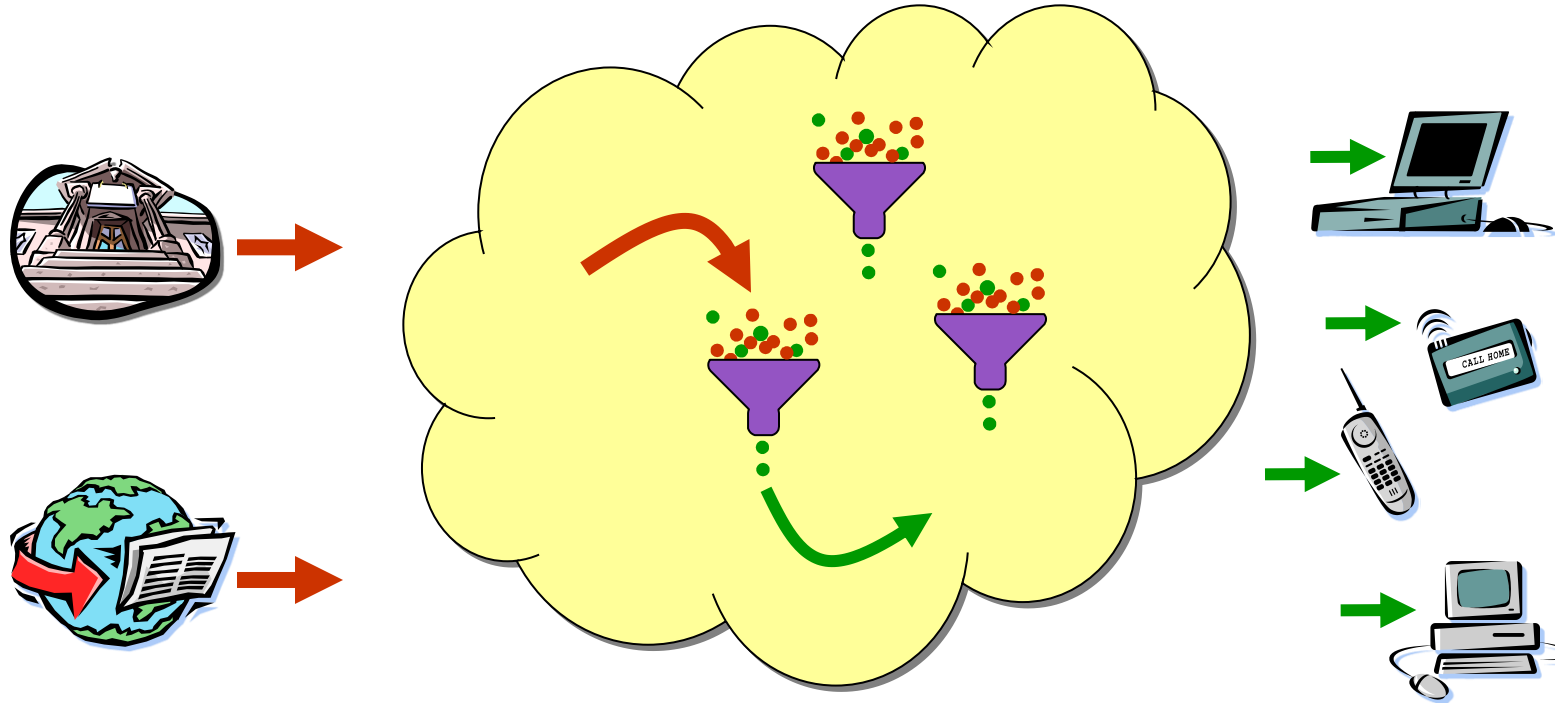
Some slides taken from: Adam Shook, CMSC 491 - Hadoop-Based Distributed Computing,
and Pascal Felber, University of Neuchatel

The publish/subscribe paradigm

Publishers: producers of information
(e.g., stock quote, news feeds...)

Brokers: identify events that match consumer interests

Subscribers: consumers of information

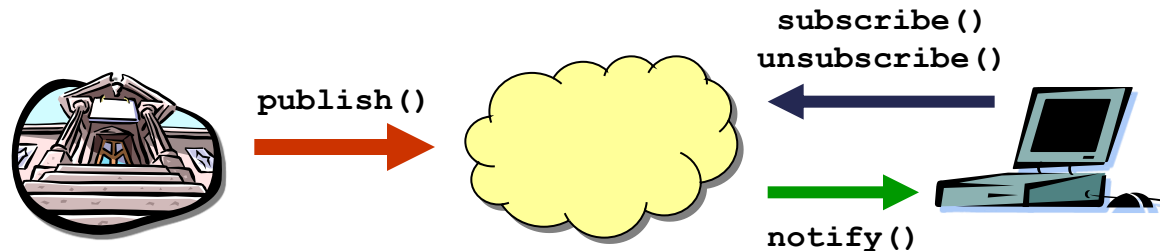


Publish/subscribe middleware

Centralized vs. distributed, persistent (DB) vs. transient,
topic- vs. content-based, reliable vs. best-effort, etc.

Publish/subscribe benefits

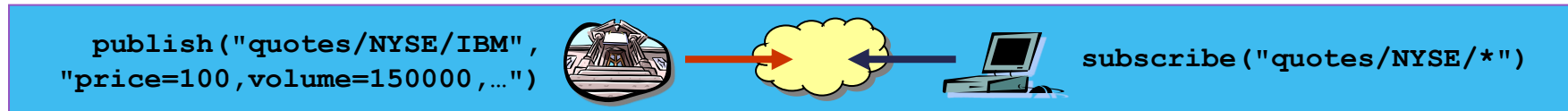
- **Space**: consumers and producers do not need to know each other
- **Time**: consumers and producers do not need to participate at the same time
- **Synchronization**: producers are not blocked, consumers receive messages asynchronously
- **Indirect addressing** based on data and subscriptions



Publish/subscribe addressing

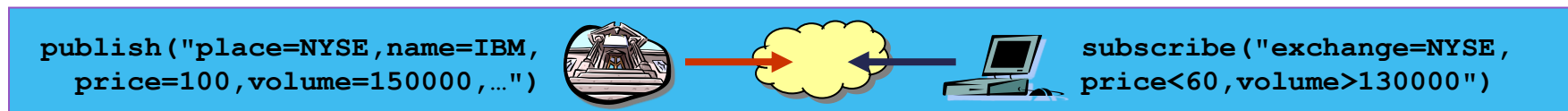
- **Topic-based P/S**

- Named subscriber groups (like newsgroups, GC)
- Often topic hierarchies and wildcard-based subscriptions (subtree, tree level, etc.)



- **Content-based P/S**

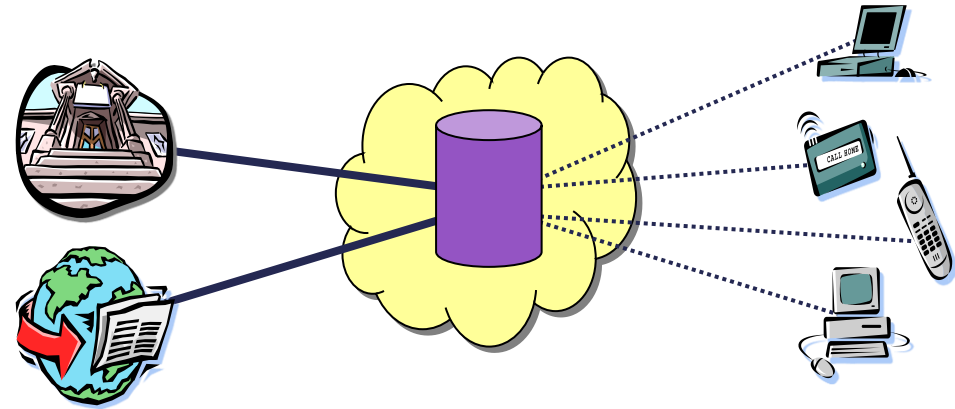
- Subscription = filter on content of messages
- Very powerful (but complex matching algorithms)



Publish/subscribe architectures

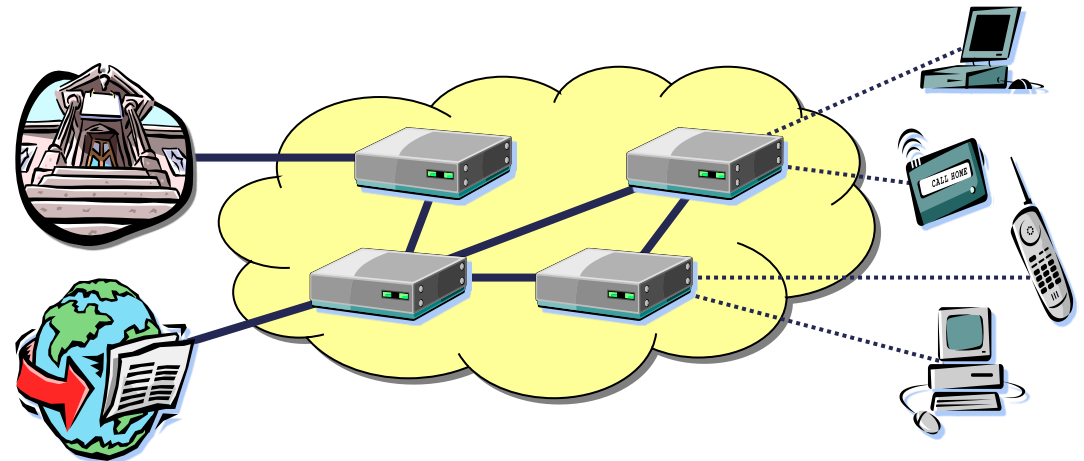
Centralized

- Single matching engine
- Limited scalability



Broker overlay

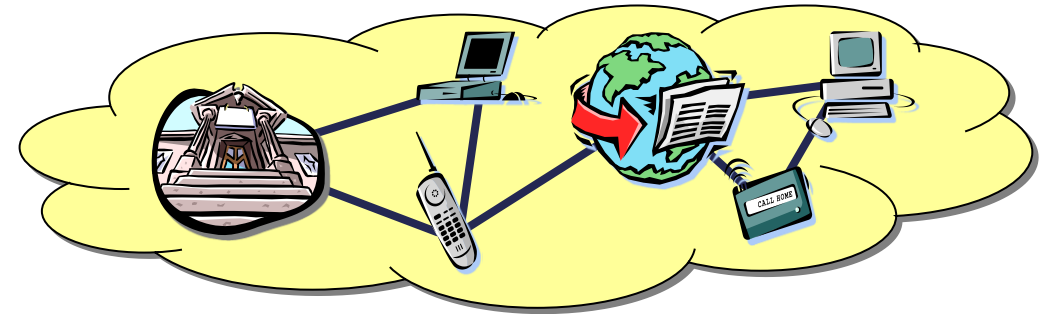
- Multiple P/S brokers
- Participants connected to some broker
- Events routed through overlay



Publish/subscribe architectures

Peer-to-peer

- Publishers & subscribers connected in P2P network
- Participants collectively filter/route events, can be both producer & consumer



Apache Kafka



- Originated at LinkedIn, open sourced in early 2011

Implemented in Scala, some Java, broker overlay

- <http://kafka.apache.org/>
- APIs for several languages

LinkedIn's motivation for Kafka was:

- “A unified platform for handling all the real-time data feeds a large company might have”

Kafka capabilities

- **Publish and subscribe** to streams of records, similar to a message queue or enterprise messaging system
- **Store** streams of records in a fault-tolerant durable way
- **Process** streams of records **as they occur**

Kafka is generally used for:

- Building real-time **streaming data pipelines** that reliably get data between systems or applications
- Building real-time **streaming applications** that transform or react to the streams of data

Kafka adoption and use cases

- Netflix: real-time monitoring and event processing
- Twitter: as part of their Storm real-time data pipelines
- Six of the top 10 travel companies, seven of the top 10 global banks, eight of the top 10 insurance companies, and nine of the top 10 US telecom companies use Kafka
- In production by more than a third of the Fortune 500

Kafka at LinkedIn

Everything in the company is a real-time stream

- > 500 billion messages written per day
- > 2.5 trillion messages read per day
- ~ 1 PB of stream data

Tens of thousands of producer processes

Backbone for data stores

- Search, Social Graph, Newsfeed, Storage

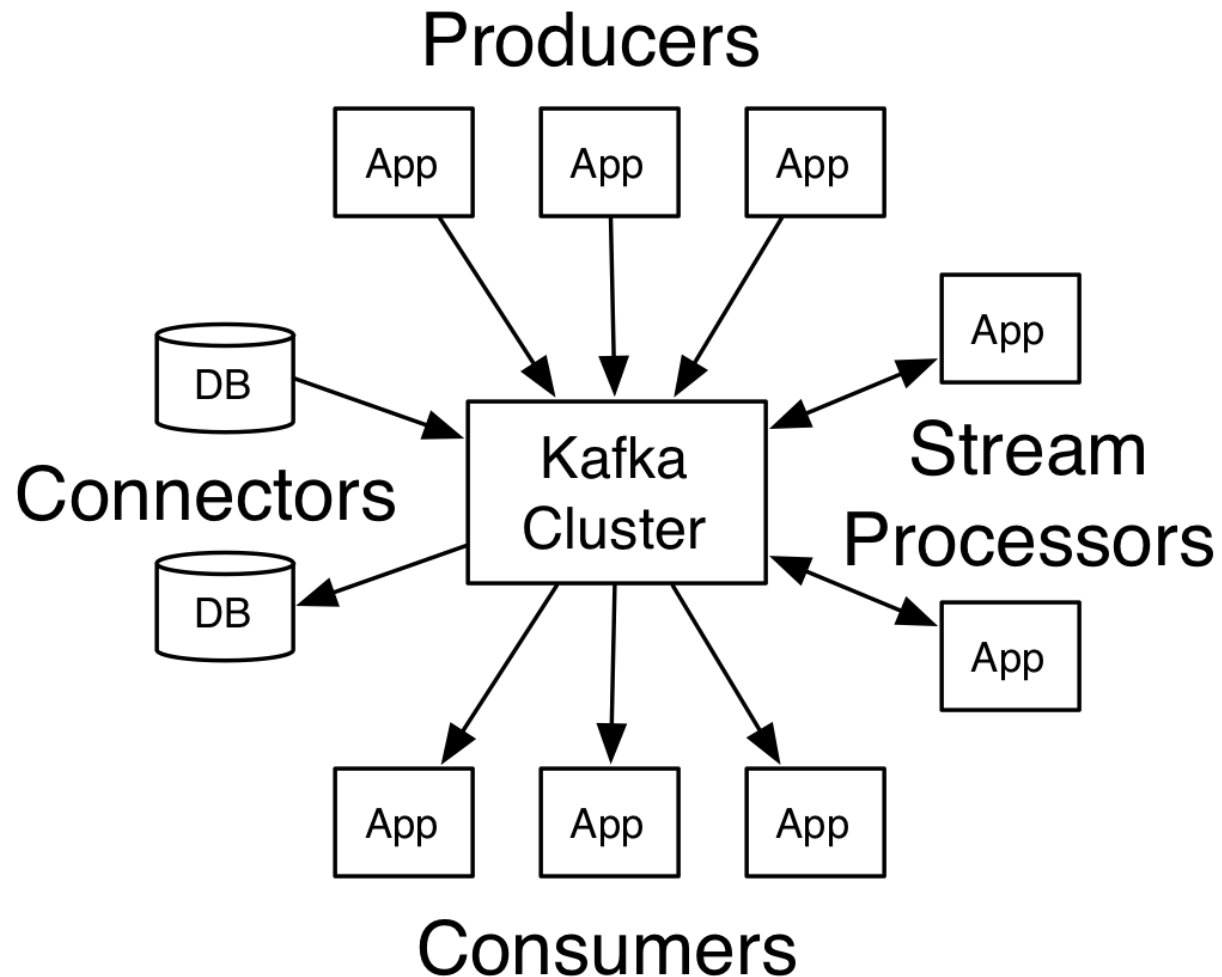
Kafka

- Kafka is run as a **cluster** on one or more servers that can span multiple datacenters
- The Kafka cluster stores **streams of records** in categories called **topics**

Each **record** consists of:

< a **key**, a **value**, and a **timestamp** >

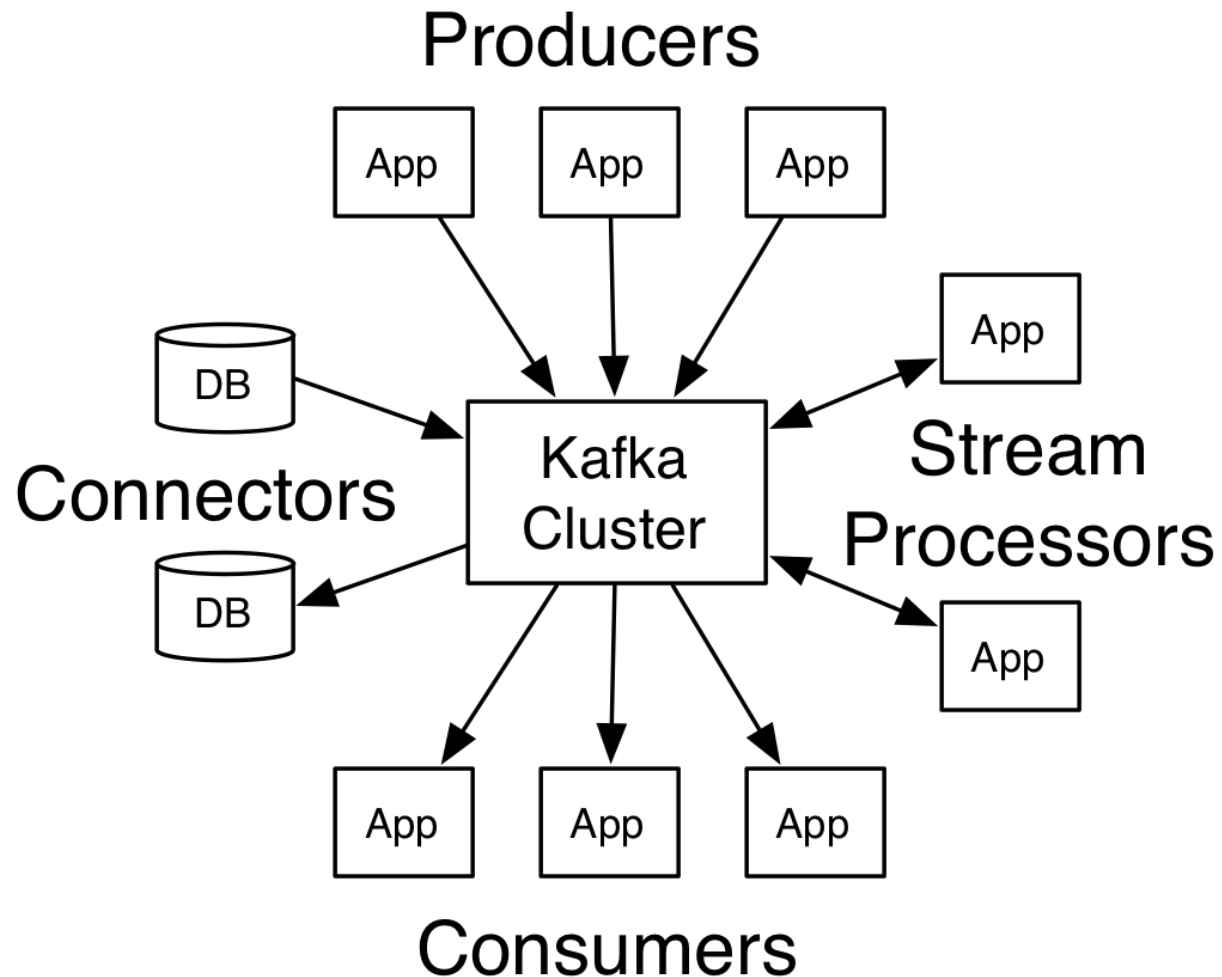
Kafka



Connectors connect Kafka topics to existing applications

- For example, a connector to a relational database might capture every change to a table

Kafka



Kafka uses **Zookeeper** for maintaining configuration information, naming, providing distributed synchronization, and providing group services

Kafka features

- High throughput to support high volume feeds
- Real-time processing of feeds to create new, derived feeds
- Fault-tolerance in the presence of machine failures
- All data persistent (until a time-to-live)
- Messages strictly ordered
- Distributed: partitioning and replication

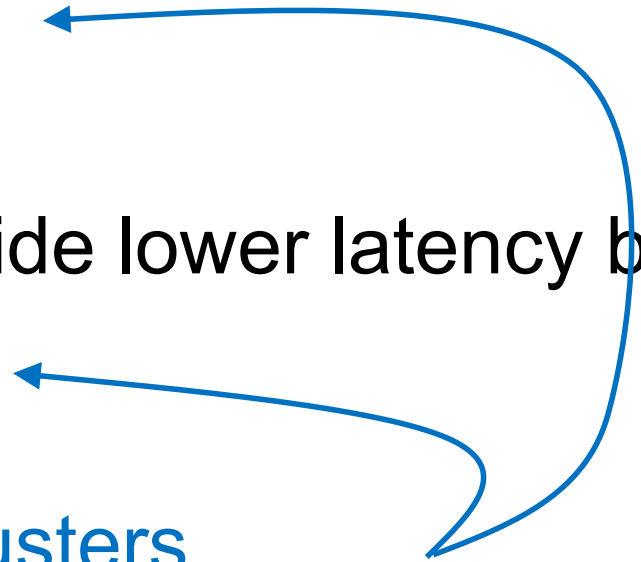
Latency vs throughput tradeoff

Kafka's strength is **high producer throughput, not small latency**
(from producer to consumer)

- Kafka processes messages in **batches**
- Typical latency is 1-2 seconds
- Throughput: some 100k per second

Other frameworks like MQTT brokers provide lower latency but less throughput (some 10k per second)

Higher throughput achieved using large clusters



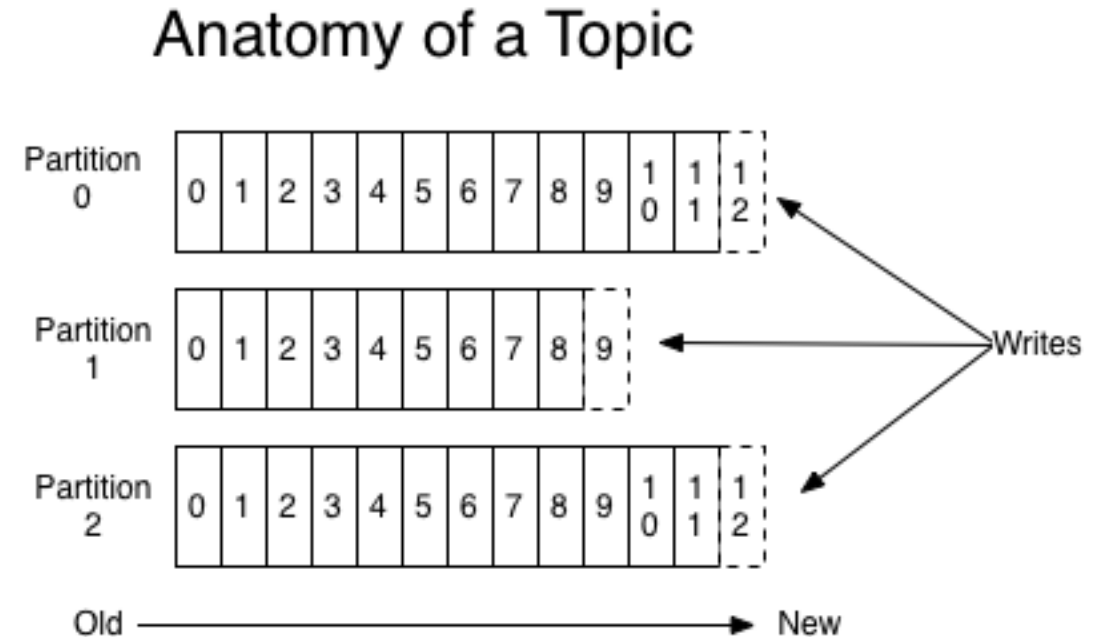
Topics and Logs

Topic: a stream of records of a given category
(the topic name)

Each topic is written to a
partitioned log

Partition: ordered sequence of
records continually appended to

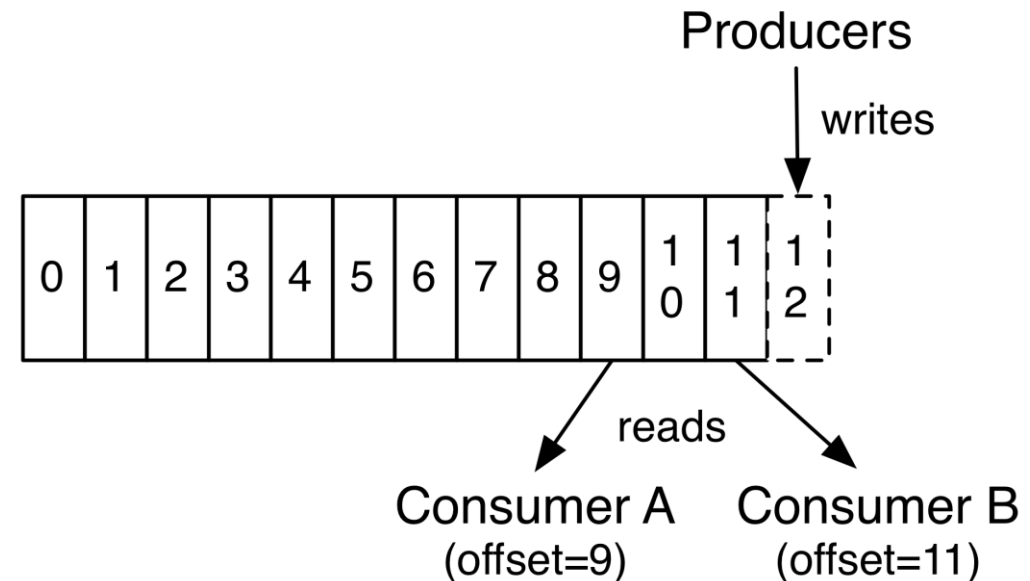
Each record is assigned
a sequential id number: the **offset**



Topics and Logs

Kafka durably persists all published records using a configurable retention period

- The only metadata retained on a per-consumer basis is the **offset of that consumer** in the log
- The offset is **controlled by the consumer**



Producers

Producers publish data to the topics of their choice

- The producer is **responsible for choosing** which record to assign to **which partition** within the topic
- Optionally consumers may be registered to a single partition
- This can be done in a **round-robin** fashion simply to balance load or according to some **semantic** partition function

Consumers

Consumers label themselves with a **consumer group name**

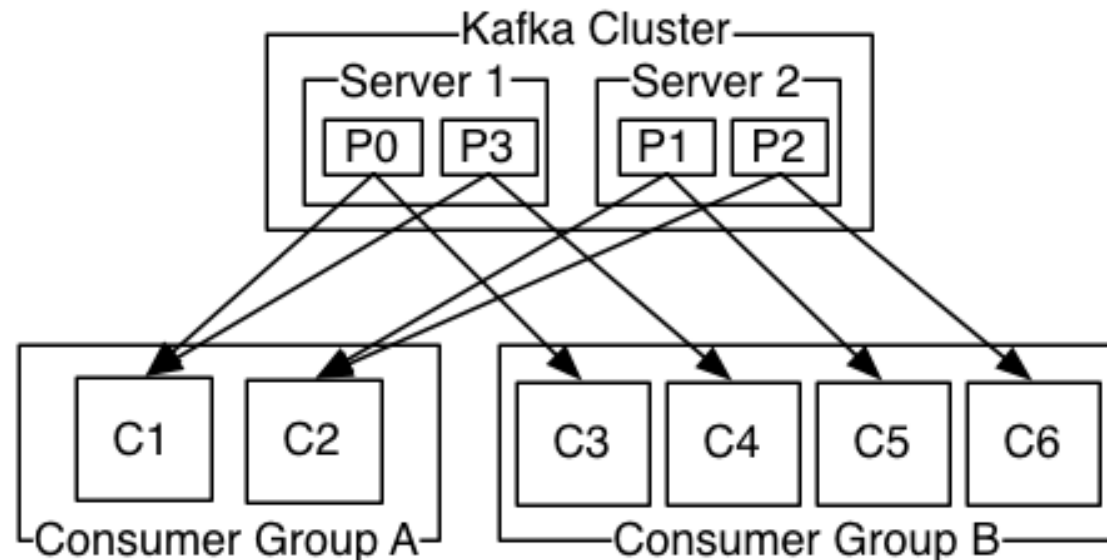
Each record published to a topic is **delivered to one consumer instance within each subscribing consumer group**

- If all the consumer instances have the same consumer group, then the records will effectively be load balanced over the consumer instances
- If all the consumer instances have different consumer groups, then each record will be broadcast to all the consumer processes

Consumers

Commonly, topics have a small number of consumer groups, one for each "logical subscriber"

- Each group is composed of many consumer instances for **scalability and fault tolerance**

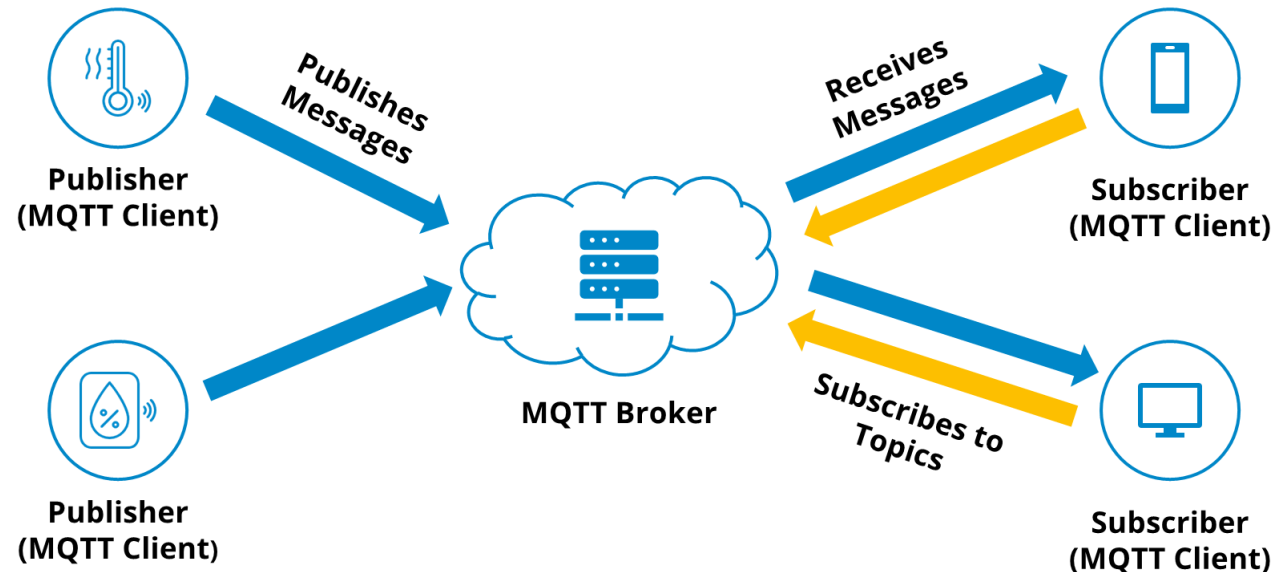


Kafka as a stream processor

- A Kafka stream processor takes continual data **streams from input topics**, performs some **processing**, and produces continual **streams of data to output topics**
- Ex.: a retail application takes in input streams of sales and shipments, and outputs a stream of price adjustments computed off this data
- Kafka supports complex transformations and **JOINing streams together**

MQTT: lightweight publish/subscribe

- MQTT is a **lightweight publish/subscribe** messaging protocol designed for constrained devices and unreliable networks
- It is widely used in IoT, smart homes, industrial telemetry, and **edge systems**



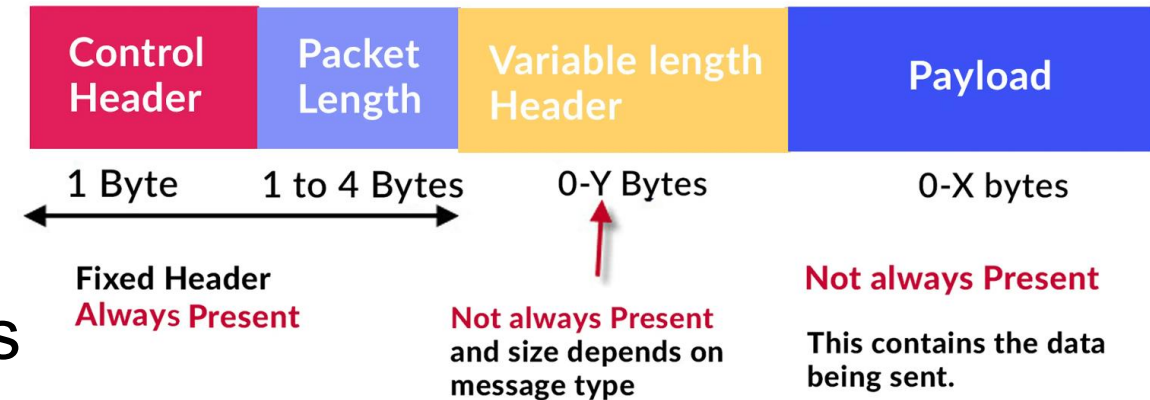
MQTT = Message Queue Telemetry Transport

MQTT protocol overview

- MQTT runs over **TCP/IP** on port 1883 (plain) or 8883 (TLS)
- Messages use a compact binary format with a small fixed header and an **application-defined payload** (JSON, binary, Protobuf, etc.)
- Notable properties:
 - Persistent TCP connection with keep-alive
 - Minimal protocol overhead
 - Optional authentication and TLS encryption
 - Well-suited for **low-bandwidth** and **high-latency networks**

Why MQTT works well on constrained networks

1. Compact binary header of few bytes
2. QoS 0 for loss-tolerant applications
3. Persistent TCP connection: there is no per-message handshake
4. The client sends a PINGREQ only when no other traffic is flowing - this allows disconnection detection without wasting bandwidth
5. Application-defined payload MQTT - compact binary encoding can be used instead of verbose JSON



MQTT topics and subscriptions

- Topics are **hierarchical** strings separated by /
- Publishers send to one topic; subscribers receive all messages whose topic matches their **filter**
 - Example topic: `texthome/livingroom/temperature`
 - Wildcard subscriptions: `texthome/+/temperature`
- Unlike Kafka, MQTT topics are **not durable logs**
- Messages are delivered to **currently connected** subscribers only

MQTT Quality of Service

Three QoS levels

- The publisher sets a QoS level per message
- The subscriber sets a maximum QoS at subscription time
- The broker delivers at the lower of the two

QoS	Guarantee	Mechanism	Risk
0	At most once	Fire and forget	Message may be lost
1	At least once	ACK-based	Duplicates possible
2	Exactly once	4-step handshake	Highest overhead

Higher QoS = stronger delivery guarantee, but also more messages, latency, and state maintained by broker and client

Mosquitto: an MQTT broker



- A lightweight open-source MQTT broker

Mosquitto vs. Kafka at a glance:

	Mosquitto / MQTT	Kafka
Model	Broker-based routing	Durable distributed log
Best for	IoT, low-power devices	High-throughput event streaming
Message retention	Ephemeral (by default)	Persistent, replayable

EMQX: a scalable MQTT broker

- Renowned for its robust, scalable architecture and rich enterprise features
- Well suited for production deployments, cloud, and industrial IoT



EMQX

Language

Erlang

Max
connections

Up to 100 million clients
in a single cluster

Clustering

Native support via CLI,
HTTP API, and
dashboard

Deployment

Cloud, edge, AWS, GCP,
Azure

EMQX

Features:

- Sub-millisecond latency; it processes millions of messages per second
- TLS/SSL, JWT, PSK, and X.509 authentication
- Built-in dashboard - Integrates with Kafka, databases, and cloud services

	Mosquitto	EMQX
Architecture	Single-threaded, C	Distributed, Erlang/OTP
Footprint	<1 MB	~50 MB
Clustering	No	Yes
Best for	Edge, prototypes, teaching	Production, enterprise, IoT at scale

Mosquitto and EMQX are complementary: Mosquitto at the edge, EMQX as the central broker
- Many real-world architectures use both

Kafka quickstart - Docker

How to install Kafka on Docker:

- <https://www.conduktor.io/kafka/how-to-start-kafka-using-docker/>

Import Kafka and Zookeeper:

- Go to the GitHub project: <https://github.com/conduktor/kafka-stack-docker-compose>
- Download the project:
 - `git clone https://github.com/conduktor/kafka-stack-docker-compose.git`

Kafka quickstart - Docker

Launch a Kafka cluster with one Zookeeper and one Kafka broker (zk-single-kafka-single.yml):

From a system shell:

```
docker-compose -f zk-single-kafka-single.yml up -d
```

- Docker downloads Kafka and Zookeeper packets and installs them as Containers

Kafka quickstart - Docker

Check to make sure both the services are running:

```
docker-compose -f zk-single-kafka-single.yml ps
```

Running commands from within the Kafka docker container:

```
docker exec -it kafka1 /bin/bash
```

Inside Docker Desktop enable a full-featured shell

```
[appuser@kafka1 ~]$ bash
```


Kafka quickstart - Docker

From within the container, you can start running some Kafka commands:

```
[appuser@kafka1 ~]$ kafka-topics --version
```

```
kafka-topics --create --bootstrap-server localhost:9092  
--replication-factor 1 --partitions 1 --topic TutorialTopic
```

```
echo "Hello, World" | kafka-console-producer --broker-list localhost:9092  
--topic TutorialTopic
```

```
kafka-console-consumer --bootstrap-server localhost:9092  
--topic TutorialTopic --from-beginning
```

Kafka and Python

Kafka has libraries for several languages

Main ones for python: kafka-python,
confluent-kafka-python

pip install kafka-python

Kafka and Python: Producer

```
from time import sleep
from json import dumps
from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers=['localhost:9092'],
                          value_serializer=lambda x:
                          dumps(x).encode('utf-8'))

for e in range(1000):
    data = {'number' : e}
    producer.send('numtest', value=data)
    print(str(data)); sleep(1)
```

Kafka and Python: Consumer

```
from kafka import KafkaConsumer
from pymongo import MongoClient
from json import loads
```

← writes to
MongoDB

```
consumer = KafkaConsumer(
    'numtest',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='earliest',
    enable_auto_commit=True,
    group_id='my-group',
    value_deserializer=lambda x: loads(x.decode('utf-8')))
```

Kafka and Python: Consumer (cont'd)

connect to MongoDB server

```
client = MongoClient('localhost:27017')
```

```
collection = client.numtest.numtest
```

```
for message in consumer:
```

```
    message = message.value
```

add the message as a document

to the 'numtest' collection of 'numtest' database

```
    collection.insert_one(message)
```

```
    print('{} added to {}'.format(message, collection))
```